IT4010 Research Project – 2026 Jul

Topic Assessment Form

V2.2

Project ID: J26-SE-311

1. Topic (12 words max)

Dependency Aware Failure Domain Risk Assessment For Kubernetes Deployed Microservice Architectures

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	
Year	

5. Brief description of the research problem including references (300 words max) – references not included in word count.

Kubernetes is widely used to deploy microservice-based systems because it supports replication, scheduling, and recovery. However, having multiple replicas does not guarantee that an end-to-end business workflow can survive infrastructure failure. Kubernetes provides placement controls such as node affinity and topology spread constraints, but these operate mainly at deployment level rather than business-workflow level [1] [2]. If all replicas of a mandatory microservice are placed on the same worker node, the failure of that node can make the service unavailable and disrupt several dependent workflows, even when most other services remain healthy.

This creates a gap between Kubernetes-level availability and workflow-level resilience. Kubernetes can show pod health, replica count, node placement, and disruption-related information [3], but it does not directly explain which business workflows will fail when a particular worker node becomes unavailable. Similarly, replica-count or deployment-only analysis may overestimate resilience because it ignores logical service dependencies.

The problem affects teams responsible for cloud-native reliability, especially when they need to evaluate whether current deployment topology protects critical operations such as ordering, payment, refund, or confirmation workflows. Prior work shows that Kubernetes availability depends on deployment architecture, redundancy, and failure

conditions [4]. However, the workflow-level impact of actual pod placement remains insufficiently addressed.

This research addresses the problem by combining business workflow dependencies with actual Kubernetes pod-to-node placement. The proposed system predicts service survival, workflow survival, and workflow-level blast radius for single worker-node failures, then validates those predictions through controlled failure-injection experiments.

6. Brief description of Existing Research and Systems (300 words max)

Kubernetes already provides mechanisms that support application availability, including Deployments, ReplicaSets, readiness probes, affinity and anti-affinity rules, topology spread constraints, and PodDisruptionBudgets [1] [2] [3]. Observability and fault-injection tools such as OpenTelemetry and Chaos Mesh can also help collect runtime evidence and test system behaviour under failure [5] [6].

Existing research has studied Kubernetes availability, microservice resilience, service placement, dependency graphs, scheduling, and fault-injection-based evaluation. These studies show that deployment topology and failure conditions can strongly influence system availability [4], and that dependency graphs and simulation can support resilience analysis [7]. Recent failure-injection research also demonstrates the value of controlled experiments for evaluating Kubernetes resilience [8]. However, many existing approaches focus mainly on service-level or infrastructure-level outcomes. They may identify failed pods, unavailable services, or replica concentration, but they do not always explain which complete business workflows are affected.

A deployment analyser can detect that all replicas of a service are placed on one worker node, but it may not know that the service is mandatory for several workflows. A dependency-only model can identify important services, but it cannot evaluate risk without knowing where replicas are actually deployed. Placement-optimization research also tends to focus on optimizing deployment decisions rather than explaining workflow-level blast radius for current Kubernetes placement [9].

The proposed research fills this gap by predicting workflow-level impact under single worker-node failures, comparing the prediction against controlled experiments, and recommending minimal transparent deployment changes.

7. Brief description of the solution's nature, including a conceptual diagram (maximum 500 words).

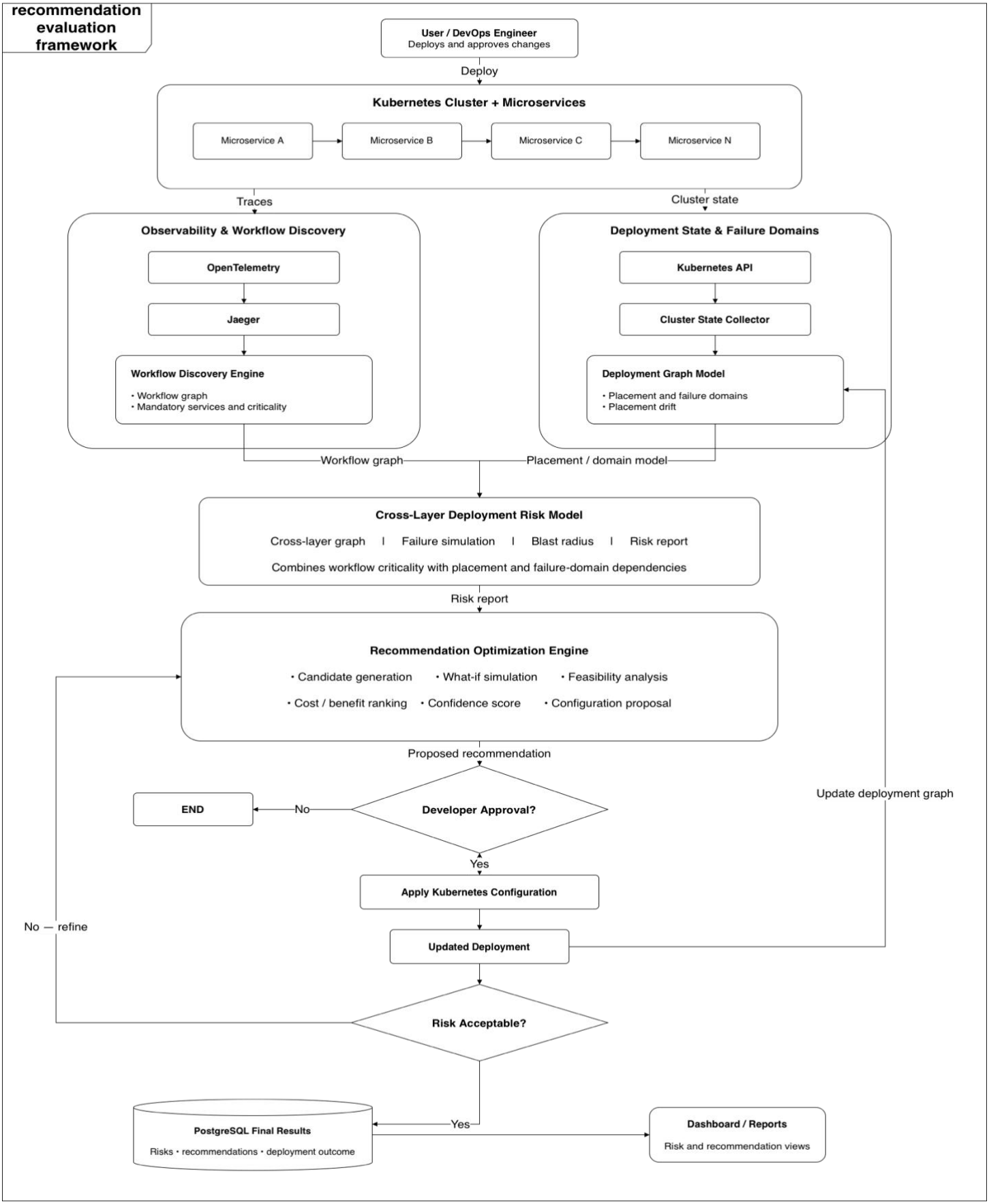
   The proposed solution is a dependency-aware failure-domain risk assessment system for Kubernetes-deployed microservice architectures. It connects logical business workflows with the physical placement of service replicas across Kubernetes worker nodes.

   The system first models each business workflow using workflow definitions and distributed traces. OpenTelemetry trace data can represent service interactions using spans, status information, and causal relationships between operations [5]. This helps identify the mandatory services required for successful workflow completion. The system then builds a deployment topology model using Kubernetes manifests and live cluster state, mapping each service to its pods, worker nodes, readiness status, and failure-domain labels. Kubernetes scheduling and topology mechanisms provide the deployment information needed for this model [1] [2].

   These two models are combined into a cross-layer graph with the structure: workflow -> service -> pod -> node -> failure domain. The analysis engine simulates the failure of one worker node by removing all pods hosted on that node from the model. It then checks whether each service still has at least one healthy reachable replica. A workflow is predicted to survive only if all mandatory services in that workflow remain available.

   The system calculates service survival, workflow survival, and node-level blast radius. It also identifies the cause of each predicted workflow failure by tracing the path from failed node to lost pod, affected service, and failed workflow. Based on this evidence, the system recommends minimal deployment changes such as redistributing an existing replica, applying pod anti-affinity, adding topology-spread constraints, or increasing replica count only when required.

   Predictions and recommendations will be validated through controlled worker-node failure experiments. Fault-injection tools such as Chaos Mesh can support repeatable failure scenarios in Kubernetes environments [6]. The evaluation will compare the proposed cross-layer method against replica-count, deployment-only, and dependency-only baselines.


recommendation evaluation framework
User / DevOps Engineer
Deploys and approves changes
Deploy
Kubernetes Cluster + Microservices
Microservice A
Microservice B
Microservice C
Microservice N
Traces
Cluster state
Observability & Workflow Discovery
OpenTelemetry
Jaeger
Workflow Discovery Engine
• Workflow graph
• Mandatory services and criticality
Deployment State & Failure Domains
Kubernetes API
Cluster State Collector
Deployment Graph Model
• Placement and failure domains
• Placement drift
Workflow graph
Placement / domain model
Cross-Layer Deployment Risk Model
Cross-layer graph | Failure simulation | Blast radius | Risk report
Combines workflow criticality with placement and failure-domain dependencies
Risk report
Recommendation Optimization Engine
• Candidate generation • What-if simulation • Feasibility analysis
• Cost / benefit ranking • Confidence score • Configuration proposal
Proposed recommendation
END
No
Developer Approval?
Update deployment graph
Yes
Apply Kubernetes Configuration
No — refine
Updated Deployment
Risk Acceptable?
PostgreSQL Final Results
Risks • recommendations • deployment outcome
Yes
Dashboard / Reports
Risk and recommendation views

8. Brief overview of the availability of necessary specialized domain expertise, knowledge, and data. (500 words max)

   The research requires knowledge of software architecture, microservices, Kubernetes deployment, distributed tracing, graph modelling, resilience testing, and experimental evaluation. These areas are supported by the Software Engineering degree programme, the Software Systems and Technologies research cluster, official Kubernetes and OpenTelemetry documentation, open-source tools, and prior research on Kubernetes availability and microservice resilience [1] [2] [3] [4] [5] [6] [7] [8].

   The work can be divided into four technical areas: workflow dependency modelling, Kubernetes deployment topology modelling, cross-layer failure-impact prediction, and minimal-change remediation with experimental validation. This division allows each member to contribute an independently evaluatable component while sharing common graph schemas, experiment scripts, and result datasets.

   The project does not require confidential organisational data or production-system access. Required data will be generated in a controlled research environment using one benchmark microservice system and one smaller validation system. Data will include workflow definitions, distributed traces, Kubernetes manifests, pod-to-node mappings, readiness states, predicted failure outcomes, observed workflow results during node-failure experiments, and before-and-after recommendation results.

   The experiment environment can be built using open-source tools such as Kubernetes, OpenTelemetry, Jaeger, Chaos Mesh or equivalent failure-injection tooling, workflow test scripts, and graph-processing libraries [5] [6]. Synthetic accounts and non-sensitive test data will be used. Since the research evaluates software behaviour and infrastructure resilience without human participants or sensitive personal data, ethical risk is low, subject to university confirmation.

9. Objectives and Novelty

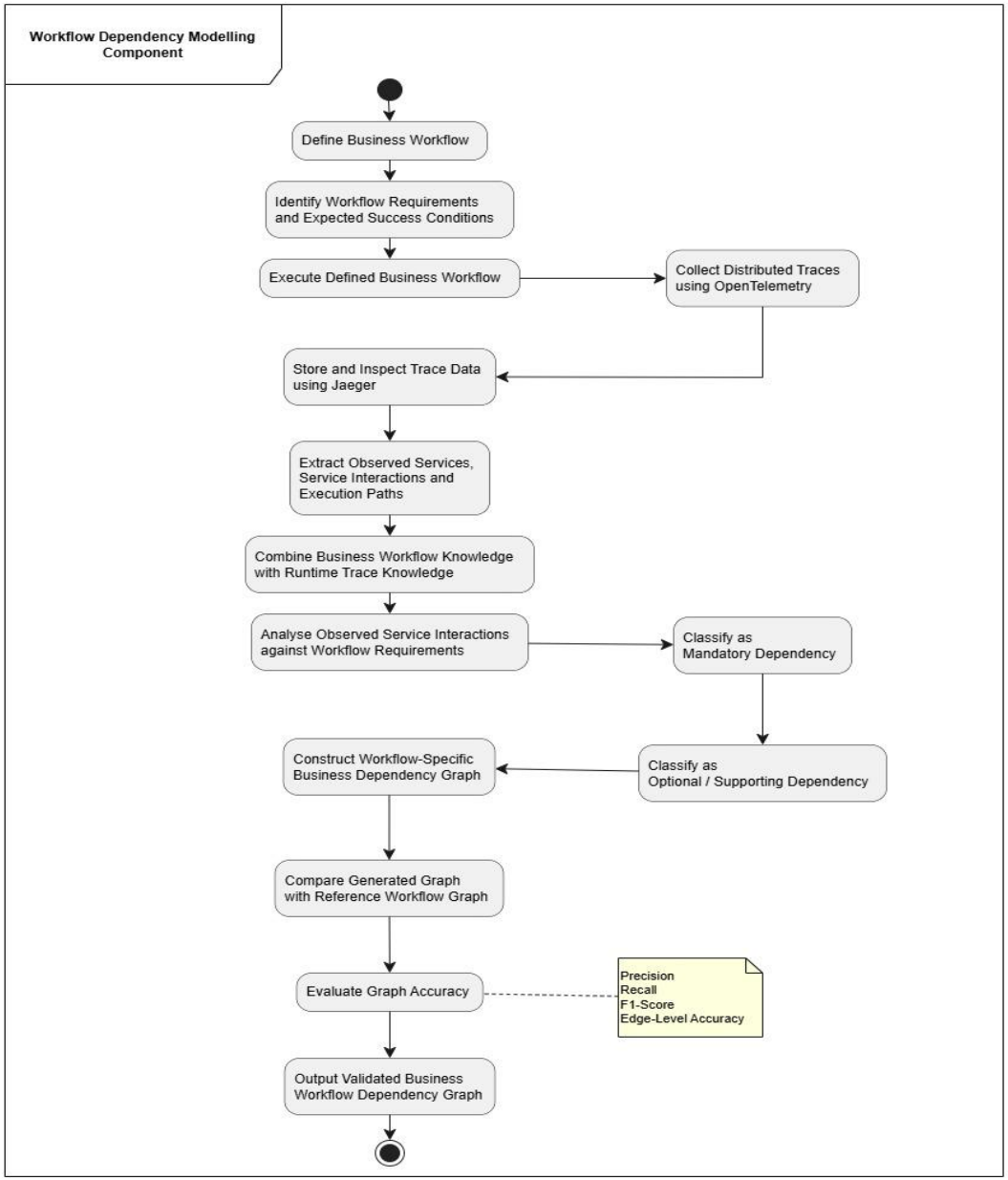
Main Objective *To design and experimentally evaluate a dependency-aware cross-layer risk assessment system that combines business workflow dependencies with Kubernetes pod placement to predict service survival, workflow survival, and blast radius under single worker-node failures, and to recommend minimal deployment changes that improve microservice workflow resilience.*			
Member Name with Registration No	Sub Objective	Tasks	Novelty
A.D. Perera – IT23236400	Identify and validate the mandatory service dependencies required for defined business workflows by combining workflow definitions with runtime distributed traces.	1.Define workflows and expected success conditions 2.Collect business workflow knowledge 3.Execute workflows and collect runtime traces using **OpenTelemetry** 4.Store and inspect traces using **Jaeger** 5.Analyse observed service interactions against workflow requirements 6.Classify dependencies as **mandatory** or **optional/supporting** 7.Construct workflow dependency graphs 8. Validate the generated graphs against reference workflows using **precision, recall, F1-score, and edge-level accuracy**	Combines **business workflow definitions** with runtime distributed traces to identify and validate which service dependencies are **truly mandatory** for workflow completion, rather than treating all observed service interactions as equally critical.

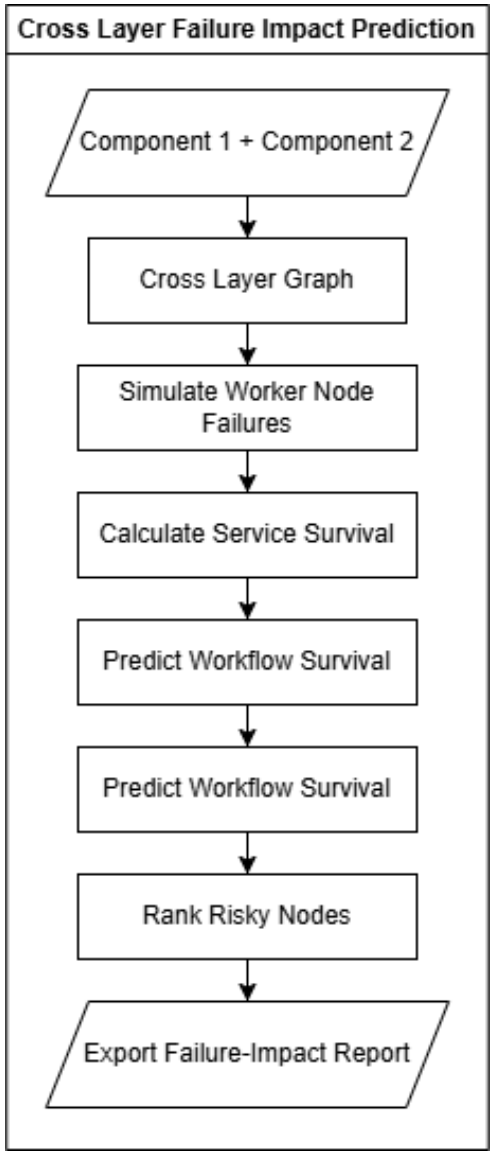
P.A.M. Minara – IT23154476	To construct and validate a Kubernetes deployment-topology model that compares declared deployment intent with actual runtime pod placement in order to identify service-level replica concentration and single-worker-node failure-domain exposure.	• Parse Kubernetes manifests (Deployments, Services, ReplicaSets) to extract declared replicas, selectors, labels, readiness probes, node affinity, pod anti-affinity, and topology spread constraints. • Query live cluster state via the Kubernetes API to collect actual pod phase, readiness, restart count, node assignment, and owner references. • Resolve the Service → Pod → ReplicaSet → Deployment → Node ownership chain to build an accurate service-level topology model. • Detect replica concentration by checking whether all ready replicas of a service share the same failure domain . • Calculate service-level failure-domain exposure – i.e. which services would lose all ready replicas if a specific worker node failed. • Compare declared intent against **runtime placement across initial deployment, scale-**	Validates the gap between declared Kubernetes deployment intent and actual runtime pod placement by detecting replica concentration, quantifying service-level failure-domain exposure, and analysing placement drift under changing runtime conditions such as scaling, rollouts, and rescheduling.

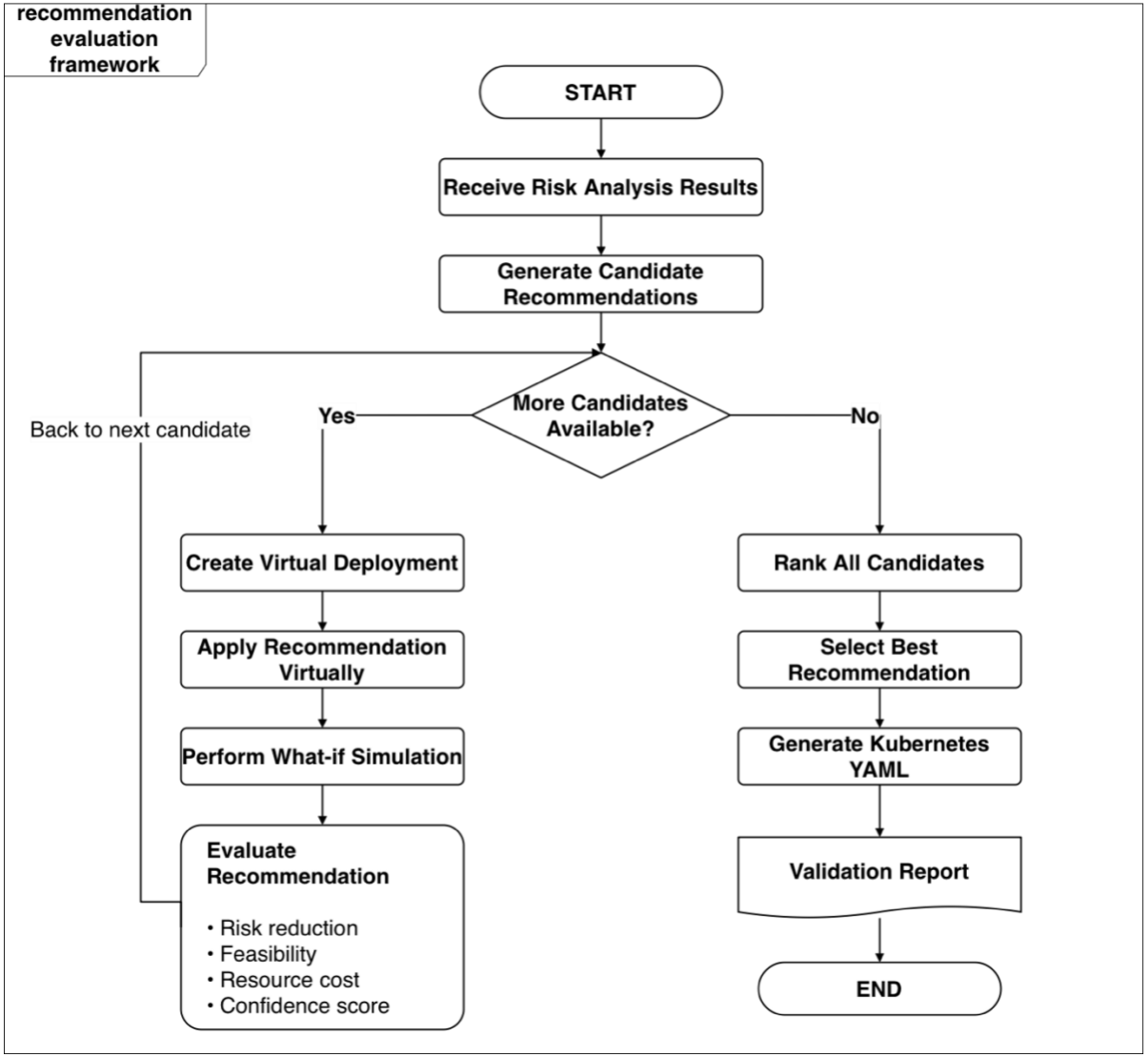
		up/down, rolling update, pod deletion/reschedule, and node cordon/drain scenarios to analyse placement drift. • Export a validated DeploymentGraph (JSON) containing services, pods, nodes, readiness, concentration status, and exposed failure domains for Component 3.	
K.V.D.Y.R. Karawita – IT23236882	Predict service survival, workflow survival, and node-level blast radius using a cross-layer workflow-deployment model.	• **Construct the Cross-Layer Graph** (Component 1 + 2) • **Simulate Worker-Node Failures** - Evaluate failure scenarios by virtually removing all pods hosted on a selected worker node and observing the resulting system state. • **Calculate service survival** - Determine whether each service remains operational after a node failure by checking if at least one healthy replica still exists. • **Predict Workflow Survival** - Identify whether a business workflow can continue functioning based on the	Deterministic cross-layer graph simulation method that converts service-level deployment exposure into workflow-level failure-impact prediction by combining mandatory workflow dependencies with actual Kubernetes pod placement.

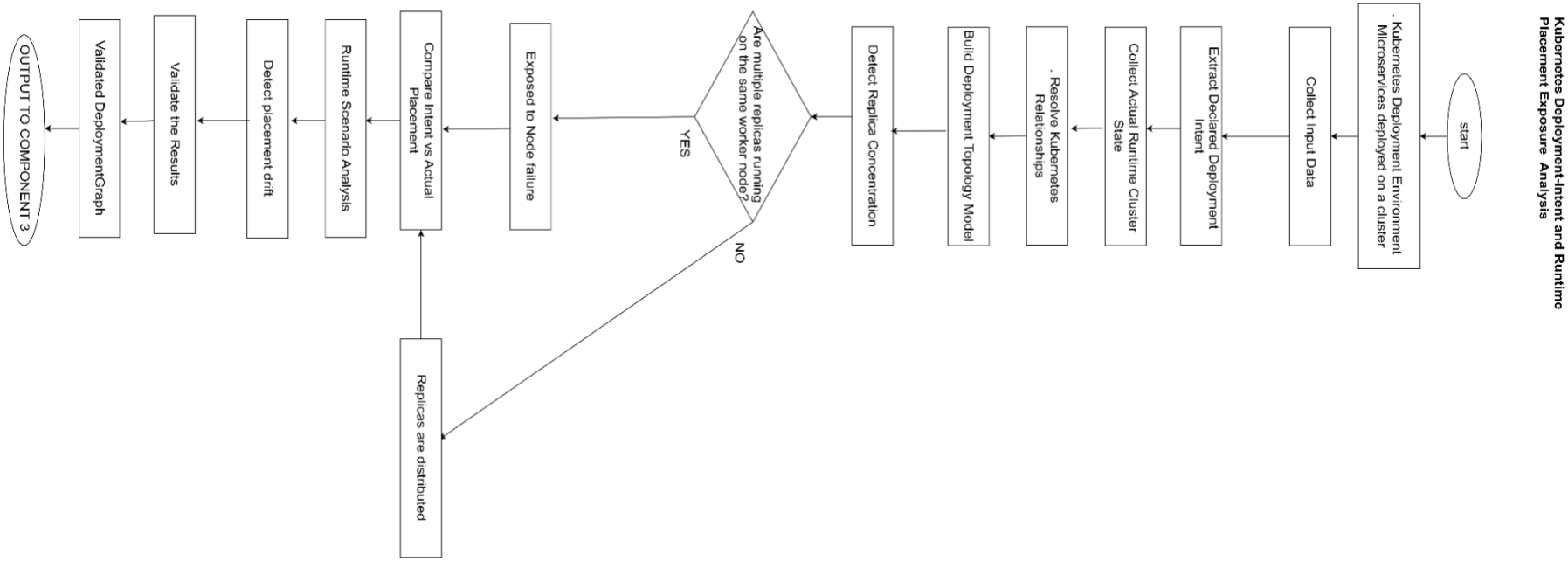
		availability of all its mandatory services. • **Compute Node-Level Blast Radius** - Measure the business impact of a node failure by calculating how many workflows are predicted to fail. • **Rank Risky Worker Nodes** - Prioritize worker nodes according to the severity of workflow disruption they can cause if they fail. • **Generate Failure-Impact Reports** (JSON Format) - Produce detailed reports showing service failures, workflow failures, blast radius, and node risk rankings for Component 4.	
L.A. Nemal – IT23236264	Develop a context-aware recommendation evaluation framework for improving Kubernetes deployment resilience.	• Generate candidate recommendations using context-aware rules. • Perform what-if simulation for each candidate. • Evaluate recommendation feasibility. • Estimate resilience improvement.	Introduces a **context-aware recommendation evaluation framework** that combines cross-layer risk information, virtual what-if simulation, multi-criteria evaluation, and post-deployment validation to identify and verify the most effective resilience strategy. Unlike static rule-based approaches, the framework evaluates recommendation

	.	• Rank recommendations using multiple evaluation metrics. • Generate Kubernetes configuration recommendations. • Validate recommendation effectiveness using post-deployment observations.	effectiveness before and after deployment.


Workflow Dependency Modelling Component
Define Business Workflow
Identify Workflow Requirements and Expected Success Conditions
Execute Defined Business Workflow
Collect Distributed Traces using OpenTelemetry
Store and Inspect Trace Data using Jaeger
Extract Observed Services, Service Interactions and Execution Paths
Combine Business Workflow Knowledge with Runtime Trace Knowledge
Analyse Observed Service Interactions against Workflow Requirements
Classify as Mandatory Dependency
Classify as Optional / Supporting Dependency
Construct Workflow-Specific Business Dependency Graph
Compare Generated Graph with Reference Workflow Graph
Evaluate Graph Accuracy
Precision
Recall
F1-Score
Edge-Level Accuracy
Output Validated Business Workflow Dependency Graph


Cross Layer Failure Impact Prediction
Component 1 + Component 2
Cross Layer Graph
Simulate Worker Node Failures
Calculate Service Survival
Predict Workflow Survival
Predict Workflow Survival
Rank Risky Nodes
Export Failure-Impact Report


recommendation evaluation framework
START
Receive Risk Analysis Results
Generate Candidate Recommendations
More Candidates Available?
Yes
No
Back to next candidate
Create Virtual Deployment
Apply Recommendation Virtually
Perform What-if Simulation
Evaluate Recommendation
• Risk reduction
• Feasibility
• Resource cost
• Confidence score
Rank All Candidates
Select Best Recommendation
Generate Kubernetes YAML
Validation Report
END


Kubernetes Deployment-Intent and Runtime Placement Exposure Analysis
start
. Kubernetes Deployment Environment Microservices deployed on a cluster
Collect Input Data
Extract Declared Deployment Intent
Collect Actual Runtime Cluster State
. Resolve Kubernetes Relationships
Build Deployment Topology Model
Detect Replica Concentration
Are multiple replicas running on the same worker node?
YES
NO
Exposed to Node failure
Compare Intent vs Actual Placement
Runtime Scenario Analysis
Detect placement drift
Validate the Results
Validated DeploymentGraph
OUTPUT TO COMPONENT 3
Replicas are distributed

References

[1] K. Authors, "Pod Topology Spread Constraints," 27 October 2025. [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/.

[2] K. Authors, "Assigning Pods to Nodes," 10 February 2026. [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/.

[3] K. Authors, "Disruptions," 16 May 2025. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/disruptions/.

[4] L. A. Vayghan, M. A. Saied, M. Toeroe and F. Khendek, "Kubernetes as an Availability Manager for Microservice Applications," *arXiv preprint arXiv:1901.04946,* 2019.

[5] O. Authors, "Traces," 14 January 2026. [Online]. Available: https://opentelemetry.io/docs/concepts/signals/traces/.

[6] C. Mesh, "Chaos Mesh Overview (Version 2.8.3)," 2026. [Online]. Available: https://chaos-mesh.org/docs/.

[7] A. A. Krasnovsky and A. Zorkin, "Model Discovery and Graph Simulation: A Lightweight Alternative to Chaos Engineering," *arXiv preprint arXiv:2506.11176,* 2025.

[8] Z. Chen, M. Goudarzi and A. N. Toosi, "Resilience Evaluation of Kubernetes in Cloud-Edge Environments via Failure Injection," *arXiv preprint arXiv:2507.16109,* 2025.

[9] F. Zhang, Y. Chen, H. Lu and Y. Huang, "Network-Aware Reliability Modeling and Optimization for Microservice Placement," *arXiv preprint arXiv:2405.18001,* 2024.

SLIIT UNI
THE KNOWLEDGE UNIVERSITY

IT4010 Research Project – 2026 Jul

Topic Assessment Form

V2.2

10. This section is to be completed by the Supervisor and the Co-supervisor of the Project.

a) This research topic possesses a comprehensive scope suitable for a final-year project.

Yes	✓	No	

b) The proposed topic exhibits novelty.

Yes	✓	No	

c) The student group can successfully execute the proposed project.

Yes	✓	No	

d) The proposed sub-objectives reflect the students' areas of specialization.

Yes		No	

e) Data required for the project is available.

Yes		No	

f) Any other comments:

Update the document as advised.

	Title	First Name	Last Name	Signature
Supervisor*	Mr.	Vishan	Jayasinghearachchi	
Co-Supervisor*				
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				

*- At this stage, identifying the Co-Supervisor is compulsory. Students should reach out to a staff member, with guidance from their supervisor, to serve as the co-supervisor for this project.